

## AIStudio — File Guide

---

*Version: Beta | Updated: 2026-06-24 Created: 2026-05-20 | Last updated: 2026-06-08 | Owner: Manuel Barbero*

---

### How to use this guide

---

This is a **functional/relational** file guide — answers "what does each file do, what does it consume, what does it produce, what depends on it" rather than just "where does it live."

---

### §1a File Versioning Conventions

---

AIStudio uses two versioning patterns depending on file type. Both are mandatory for any file that is deployed or tracked.

#### Shell scripts ( `.sh` )

Two lines, both required, **both must match**:

```
# Version: X.Y.Z
VERSION="X.Y.Z"
```

#### Python files ( `.py` )

Header comment block at the absolute top of the file, before all imports:

```
# Version: X.Y.Z
# Changelog: X.Y.Z — <ticket> description of change.
#           Continuation lines indented 12 spaces (aligned with description start).
# Changelog: X.Y.Z-1 — previous change (newest first).
```

Rules: - `# Version:` must match the most recent `# Changelog:` entry — single source of truth. - Entries are prepended (newest first) — never append at the bottom. - One entry per version bump. Two tickets in one bump is fine. - The header block appears before `from __future__ import annotations` and all imports.

## Class C files — version inside file, not in filename

Per §1, Class C (code/SDLC) files carry no date in the filename. The `# Version:` header is the only version signal.

## §2 The runtime layer — `src/` + `front_end/` + `data/`

What runs to answer a query.

`src/local_llm_bot/app/` — Python application

| File                                     | Function                                                  | Reads                                                                           | Produces                                                               |
|------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <code>api.py</code>                      | FastAPI HTTP layer                                        | Corpus metadata, query payload, system prompt                                   | JSON response with citations                                           |
| <code>rag_core.py</code>                 | RAG pipeline orchestration                                | Query, corpus index                                                             | Embedded query → retrieved chunks → reranked → LLM context             |
| <code>ingest/pipeline.py</code>          | Document ingestion (per-file chunked/ embedded/ upserted) | Files in <code>data/corpora/&lt;name&gt;/uploads/</code> , corpus metadata seed | <code>index.jsonl</code> , <code>manifest.jsonl</code> , Qdrant chunks |
| <code>ingest/loaders.py</code>           | File-type-specific loaders                                | PDF/DOCX/HTML/ MD files                                                         | Plain text + page metadata                                             |
| <code>ingest/chunking.py</code>          | Text chunking with overlap                                | Plain text                                                                      | Chunks with stable IDs                                                 |
| <code>vectorstore/qdrant_store.py</code> | Vector store interface                                    | Embeddings, metadata                                                            | Qdrant collections, similarity search                                  |
| <code>ollama_client.py</code>            |                                                           |                                                                                 |                                                                        |

| File                   | Function                  | Reads              | Produces                           |
|------------------------|---------------------------|--------------------|------------------------------------|
|                        | Ollama HTTP wrapper       | Model name, prompt | Embedding vectors, LLM completions |
| <code>config.py</code> | Environment-driven config | OS env vars        | Settings dict                      |

### `front_end/rag_studio.html` — UI

Single self-contained HTML file. Opens directly in browser, no server. Communicates with FastAPI backend at `localhost:8000`.

**Provides:** corpus selector, chat with citations, corpus CRUD (new/delete/rename), file upload + per-file inspect, metadata edit (description, `search_guidance`), help modal.

### `data/corpora/<name>/` — Document corpora

Each corpus is a self-contained folder:

| File                                                                      | What it is                                                                                                                                               | Who writes it                                                        |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| <code>uploads/</code>                                                     | Source documents (PDF, DOCX, etc.)                                                                                                                       | User via UI<br>Add Files                                             |
| <code>&lt;name&gt;_corpus_metadata.yaml</code>                            | Description, <code>search_guidance</code> , <code>sources[]</code> , default query settings, runtime stats                                               | Seed at create-time, updated at ingest; UI <b>Edit Corpus</b>        |
| <code>&lt;name&gt;_full_scope.yaml</code>                                 | Corpus <b>membership</b> — the firm/entity universe (the superset). At the corpus root.                                                                  | Downloader / entity-KB tools<br>( <i>bundled financial corpora</i> ) |
| <code>scopes/</code><br><code>&lt;name&gt;_&lt;stem&gt;_scope.yaml</code> | Named <b>subsets</b> of the membership (e.g. <code>lang_en</code> , <code>big_banks</code> ) — read-only selectors for download, bench, selective ingest | Hand-authored<br>( <i>optional</i> )                                 |
| <code>index.jsonl</code> , <code>manifest.jsonl</code>                    | Internal ingest bookkeeping (per-chunk / per-file records). Not user-edited; <b>internal, may be retired</b> in a future                                 | Ingest pipeline                                                      |

| File                | What it is                                                                            | Who writes it |
|---------------------|---------------------------------------------------------------------------------------|---------------|
|                     | cleanup — corpus config now lives in <code>&lt;name&gt;_corpus_metadata.yaml</code> . |               |
| <code>trash/</code> | Soft-deleted files (recoverable)                                                      | UI delete     |

Note the split: a corpus's **defining inputs** — documents, metadata, and (for the financial corpora) scope + entity KB — live under `data/corpora/<name>/`. Benchmark **question sets** live separately under `benchmarks/<name>/` (next section).

**Demo corpus** ships with the repo. **Help corpus** is built from `docs/` + key root markdown. **SEC 10-K corpus** is downloaded from SEC EDGAR via `ais_download_sec_10k` and ingested through the UI (Upload button), like any corpus. **User corpora** are created via the UI New button — private, gitignored, no special setup required.

### `benchmarks/<corpus>/` — Benchmark questions and reports

`benchmarks/<corpus>/` holds the bench **question sets** (inputs) and the **reports** they produce (outputs) — kept separate from the corpus-defining files under `data/corpora/<corpus>/`. Each corpus that has a questions file can be tested with `ais_bench`:

```
benchmarks/
  demo/
    demo_questions.yaml      ← pre-built questions for the demo corpus
    reports/                 ← timestamped benchmark reports (.md, .json, .pdf)
  sec_10k/
    sec_10k_questions.yaml   ← default question set
    sec_10k_<stem>_questions.yaml ← named alternates (e.g. sec_10k_June_2026_questions.yaml)
    reports/
  <your-corpus>/
    <your-corpus>_questions.yaml ← you write this (see format below)
    reports/
```

**Shipped evidence suite** — `benchmarks/docs/` holds the curated, user-facing benchmark suite: the four audited canonical reports ( `BENCH - Canonical Run A-D - ...`, `.md` + `.pdf`), the suite synthesis ( `BENCH - Canonical Suite - README and Synthesis` ), and `benchmarks/docs/reports/` (the raw `ais_bench` reports duplicated as provenance). This is what the top README points to as the evaluation evidence; reproduce it with `ais_bench --batch`. Distinct from the operator `BENCH - AIS - *` audit packages, which are governance artifacts under `meta/ais/bench/`.

The default set is `<corpus>_questions.yaml`; a named alternate is selected by stem ( `ais_bench --questions June_2026` → `sec_10k_June_2026_questions.yaml` ). Reports are

written per run into `reports/` under a standard timestamped name — `.md` to read, `.json` for tooling, `.pdf` to share.

**Questions file format** — create `benchmarks/<name>/<name>_questions.yaml`:

```
- topic: Topic Name
  questions:
    - id: unique_snake_case_id
      question: The exact question text sent to AISTudio.
      keywords: [term1, term2, term3] # all must appear in the answer to pass
      notes: What a correct answer looks like — which document, what content.
```

**What `ais_bench` checks** — three conditions, all must pass: 1. All `keywords` appear in the answer (case-insensitive) 2. The answer includes at least one citation 3. The model doesn't hedge with "no information available" or similar phrases

**Tips for good keywords:** 2–5 distinctive terms that prove the model retrieved the right content. Use specific concepts, entity names, or regulatory terms — not generic words.

For full details and advanced options, see `TUTORIAL.md §3.4` and `HOWTO.md`.

### `prompts/system.txt` — System prompt

Loaded by `api.py` at query time. Per-corpus `search_guidance` from `<name>_corpus_metadata.yaml` is appended dynamically.

## §3 The user-facing command surface

All commands available after running `./ais_install` from repo root.

| Command                | What it does                               | When you use it            | Reads / Produces      |
|------------------------|--------------------------------------------|----------------------------|-----------------------|
| <code>ais_start</code> | Starts FastAPI + Qdrant + Ollama; opens UI | Beginning of every session | — / running services  |
| <code>ais_stop</code>  | Stops all services cleanly                 | End of every session       | — / —                 |
| <code>ais_log</code>   |                                            |                            | service logs / stdout |

| Command                             | What it does                                                       | When you use it                   | Reads / Produces                                                                                         |
|-------------------------------------|--------------------------------------------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------|
|                                     | Tails the live backend log                                         | Debugging; watching queries       |                                                                                                          |
| <code>ais_bench</code>              | Runs benchmark suite against a corpus                              | Validating retrieval quality      | <code>benchmarks/&lt;corpus&gt;/</code><br><code>&lt;corpus&gt;_questions.yaml</code> / benchmark report |
| <code>ais_download_sec_10k</code>   | Downloads SEC 10-K filings from EDGAR                              | Setting up SEC corpus (optional)  | EDGAR / <code>data/corpora/sec_10k/uploads/</code>                                                       |
| <code>ais_help</code>               | Shows command reference                                            | Forgot a command                  | <code>ais_command_help.txt</code> / <code>stdout</code>                                                  |
| <code>ais_download_esef</code>      | Downloads ESEF iXBRL annual reports from filings.xbrl.org          | Setting up ESEF corpus            | <code>esef_banks_full_scope.yaml</code> / <code>data/corpora/esef_banks/uploads/</code>                  |
| <code>ais_ingest_esef</code>        | Ingests ESEF filings into AIStudio                                 | After downloading ESEF            | <code>uploads/</code> / Qdrant                                                                           |
| <code>ais_import_entity_kb</code>   | Builds entity KB for a corpus (GLEIF/ Wikidata identity + aliases) | After ingest; after adding a firm | <code>&lt;corpus&gt;_full_scope.yaml</code> / <code>gleif_&lt;corpus&gt;_full_entities.yaml</code>       |
| <code>ais_import_glossary_kb</code> | Builds glossary KB (BIS Basel acronym expansion for BM25)          | After ingest; corpus-wide         | static seed / <code>bis_basel_any_corpus_full_glossary.yaml</code>                                       |

| Command                          | What it does                                            | When you use it                  | Reads / Produces                                                                          |
|----------------------------------|---------------------------------------------------------|----------------------------------|-------------------------------------------------------------------------------------------|
| <code>ais_install</code>         | Installs/<br>updates user<br>commands                   | Fresh<br>install; new<br>command | <code>ais_user_commands_manifest.yaml</code> / shell<br>aliases in <code>~/ .zshrc</code> |
| <code>ais_create_shortcut</code> | Creates<br>AIStudio<br>Dock/<br>Desktop icon<br>(macOS) | Optional<br>post-install         | <code>/ ~/Desktop/AIStudio.app</code>                                                     |

## §9 The repo root layout

### User-visible (public, ships with repo)

| File / Dir                   | What it does                                                                                                                                                                  | Relationship                |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| <code>README.md</code>       | Product<br>overview                                                                                                                                                           | Entry point for new users   |
| <code>QUICKSTART.md</code>   | Install + first-<br>run guide                                                                                                                                                 | Read after README           |
| <code>HOWTO.md</code>        | Day-to-day<br>usage recipes                                                                                                                                                   | Read for "how do I X"       |
| <code>TUTORIAL.md</code>     | Guided<br>walkthroughs                                                                                                                                                        | Read after QUICKSTART       |
| <code>CONTRIBUTING.md</code> | Contribution<br>guide                                                                                                                                                         | Read for repo participation |
| <code>LICENSE</code>         | License terms                                                                                                                                                                 | Repo convention             |
| <code>Makefile</code>        | <code>make</code><br><code>install</code> ,<br><code>make test-</code><br><code>unit</code> , <code>make</code><br><code>test</code> , <code>make</code><br><code>lint</code> | Build orchestration         |

| File / Dir                                   | What it does                                 | Relationship                                       |
|----------------------------------------------|----------------------------------------------|----------------------------------------------------|
| <code>pyproject.toml</code>                  | Python project config (ruff, pytest)         | Tool config                                        |
| <code>requirements.txt</code>                | Python dependencies                          | <code>pip install -r</code>                        |
| <code>ais_install</code>                     | User command installer                       | Reads <code>ais_user_commands_manifest.yaml</code> |
| <code>ais_user_commands_manifest.yaml</code> | Auto-generated user command list             | Produced by install                                |
| <code>ais_*.sh</code> (user)                 | User commands (see §3)                       | Aliased by <code>ais_install</code>                |
| <code>prompts/system.txt</code>              | LLM system prompt                            | Loaded by <code>api.py</code>                      |
| <code>front_end/rag_studio.html</code>       | UI                                           | Opened in browser                                  |
| <code>src/</code>                            | Application source (see §2)                  | Imported by <code>api.py</code>                    |
| <code>tests/</code>                          | Test suite                                   | Run by <code>make test</code>                      |
| <code>docs/</code>                           | User documentation + help corpus sources     | Ingested into help corpus                          |
| <code>benchmarks/</code>                     | Benchmark harness + question files + reports | Run by <code>ais_bench</code>                      |
| <code>data/</code>                           | Corpus data (demo + help tracked; user       | Read by <code>rag_core.py</code>                   |



| File / Dir | What it does           | Relationship |
|------------|------------------------|--------------|
|            | corpora<br>gitignored) |              |

## §12 The file-to-action map

Quick lookup: who reads what, who writes what, what feeds into what.

| File pattern                                                              | Reads                                              | Writes                                     | Feeds into                                 |
|---------------------------------------------------------------------------|----------------------------------------------------|--------------------------------------------|--------------------------------------------|
| <code>&lt;corpus&gt;_corpus_metadata.yaml</code>                          | <code>api.py</code><br>(system prompt), UI<br>Edit | Ingest<br>pipeline, UI<br>Edit             | LLM routing,<br>search_guidance<br>display |
| <code>prompts/system.txt</code>                                           | <code>api.py</code>                                | Manual edit                                | LLM system<br>instructions                 |
| <code>data/corpora/&lt;name&gt;/uploads/*</code>                          | Ingest pipeline                                    | UI Add Files                               | Corpus chunks →<br>Qdrant                  |
| <code>data/corpora/&lt;name&gt;/index.jsonl</code>                        | ( <i>internal — see<br/>note</i> )                 | Ingest<br>pipeline                         | Per-chunk ingest<br>record                 |
| <code>benchmarks/&lt;corpus&gt;/<br/>&lt;corpus&gt;_questions.yaml</code> | <code>ais_bench</code>                             | User-<br>authored or<br>auto-<br>generated | Benchmark pass/<br>fail evaluation         |

### §12a What files define a corpus — and where they live

This is the *where* for corpora; the *how* (what builds them) is in `CODEBASE_GUIDE.md` → "How a corpus is defined, built, and used". Every corpus — the built-in ones and any you create — lives under `data/corpora/<name>/`.

## What the app reads

| File / dir                                                                         | Purpose                                                                                                                                                                   |
|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>data/corpora/&lt;name&gt;/uploads/</code>                                    | the actual documents (what gets searched)                                                                                                                                 |
| <code>data/corpora/&lt;name&gt;/&lt;name&gt;_corpus_metadata.yaml</code>           | the corpus's description, search guidance, and default query settings; read every time you ask a question                                                                 |
| <code>data/corpora/&lt;name&gt;/&lt;name&gt;_full_scope.yaml</code>                | (bundled <code>sec_10k</code> / <code>esef_banks</code> only) the corpus <b>membership</b> — which firms/entities belong to it                                            |
| <code>data/corpora/&lt;name&gt;/scopes/&lt;name&gt;_&lt;stem&gt;_scope.yaml</code> | (optional) named <b>subsets</b> of the membership (e.g. <code>lang_en</code> ) — selectors for download, benchmarking, and selective ingest                               |
| <code>benchmarks/&lt;name&gt;/&lt;name&gt;_questions.yaml</code>                   | (optional) benchmark question sets for the corpus — note these live under <code>benchmarks/</code> , not with the corpus                                                  |
| <code>data/knowledge_sources/gleif/&lt;name&gt;_entities.yaml</code>               | (bundled <code>sec_10k</code> / <code>esef_banks</code> only) company-identity (GLEIF) lookups used to keep firms apart in cross-company questions — see Tutorial Annex 1 |

So *what files define a corpus*? At minimum: the documents in `uploads/`, and the `<name>_corpus_metadata.yaml` that describes it. Membership scope, subset scopes, questions, and the GLEIF identity file are enrichment used by the bundled financial corpora.

**Where things live — the split.** A corpus's *defining* files sit together under `data/corpora/<name>/`: the documents, the metadata, and (for the financial corpora) the scope and entity KB. Benchmark *question sets* sit apart under `benchmarks/<name>/`, alongside the `reports/` they generate. Rule of thumb: **what defines the corpus** → `data/corpora/`; **how you test it** → `benchmarks/`.

## Where YOUR files go

When you create your own corpus, everything lives under `data/corpora/<your-corpus>/`:

- `uploads/` — your files. Created and filled by the **Upload** button in the UI; you don't make this folder by hand.
- `<your-corpus>_corpus_metadata.yaml` — created by the **Edit Corpus** modal when you give the corpus a description, search guidance, and default settings.

- `benchmarks/<your-corpus>/<your-corpus>_questions.yaml` — only if you choose to add benchmark questions.

To query your own portfolio — your filings, reports, decks, or PDFs — create a corpus in **Settings** → **New Corpus**, then drop your files in via **Upload**. The corpus is defined by what's in its `uploads/` folder plus the description you give it. That's everything you need.

## §15 Version history

| Version | Date       | Changes                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beta    | 2026-06-24 | Corpus-data consistency sweep (§71, §85, §12a): added membership scope ( <code>&lt;name&gt;_full_scope.yaml</code> ) + subset scopes ( <code>scopes/</code> ) to the corpus tables; stated the <b>defining-inputs</b> ( <code>data/corpora/</code> ) vs <b>question-sets</b> ( <code>benchmarks/</code> ) split explicitly; documented named question alternates by stem; softened <code>index.jsonl</code> / <code>manifest.jsonl</code> to internal-may-be-retired. |
| Beta    | 2026-06-15 | Trimmed to the AIStudio file, command, and corpus reference; SEC ingest is UI-only.                                                                                                                                                                                                                                                                                                                                                                                   |
| 1.2.0   | 2026-06-08 | Added §12a "What files define a corpus — and where they live" (corpus definition + where user portfolio files go).                                                                                                                                                                                                                                                                                                                                                    |
| 1.1.0   | 2026-05-25 | Added user-created corpus type. Added benchmarks/ subsection with questions file format and pass/fail explanation. Added benchmarks row to §12.                                                                                                                                                                                                                                                                                                                       |
| 1.0.0   | 2026-05-20 | Initial version.                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

